Contents

Fraktální výpočetní algebra dělitelů nuly — generativní operátory, paměť a paralelní algoritmy **1**
Abstrakt . . . 1
1. Výpočetní primitivum není uzel stromu, ale strukturální adresa . . . 1
2. Nulové matice neukládat; aplikovat je . . . 1
3. Inherited vs birth: jak reprezentovat skutečnou novost . . . 2
4. Algebraické indexy pro AI a stavové systémy . . . 2
5. Paměťová pyramida: VRAM / RAM / NVMe . . . 2
6. GPU paralelismus — rozdělovat podle algebraické práce . . . 2
7. Databáze a obecné stromy . . . 2
8. Higher-π jako výpočetní orákulum a optimalizační metrika . . . 3
9. Reprodukovatelnost a výkonnostní kontrakt . . . 3
10. První výpočetní syntéza . . . 4
11. Úplný typovaný kontrakt a algebraické invarianty . . . 4
12. Výpočetní jádra a nákladový model . . . 4
13. Paralelní determinismus a persistence . . . 4
14. Výpočetní experiment a regresní protokol . . . 4
15. Závěr . . . 5
Literatura . . . 5

Fraktální výpočetní algebra dělitelů nuly — generativní operátory, paměť a paralelní algoritmy

Samostatný článek z matematiky a informatiky · 25. října 2026
Autor: Vladimír Bíža
Téma: Signed algebraické orákulum, implicitní operátory, úložiště, CPU/GPU a návrh ověřitelných benchmarků.

Abstrakt

Převádíme exaktní informační struktury Paperu A na návrh runtime, v němž jsou genealogická adresa, incidence, fáze a matematické operátory rekonstruovatelné bez explicitní materializace celé vyšší algebry. Výsledkem není univerzální tvrzení, že zero-divisor algebra zrychlí libovolný program: je to konkrétní systém **implicitní dědičnost + explicitní novost + operátorově typovaná sémantika**, který může přímo sloužit pro adresovatelnou algebraickou paměť, multiresoluční GPU výpočty, řídké databáze a originální generativní datové indexy. Přikládáme samostatný Rust oracle, regresní testy a benchmarkový kontrakt.

1. Výpočetní primitivum není uzel stromu, ale strukturální adresa

Definujme $a \in [0, 2^k)$, $\chi = a \oplus b$, kořen (k_0, χ_0), ancestry slovo $u \in \{0,1\}^{k-k_0}$, orientaci a typ operátoru. Čistě algebraický vztah $e_a e_b = \varepsilon(a,b) e_{a\oplus b}$ dostaneme přes rekurzi v čase $O(k)$, prostoru $O(k)$ (při iterativním bitovém provedení $O(1)$ pomocné paměti). Dvojice assessorů má rozhodnutí nulové hrany čtyřmi voláními oracle.

Kanonický klíč: `CDAddr { level, a, b, chi, orientation, sign, ancestry, birth_level, relation_kind }`. `chi` nikdy nenahrazuje čtyři adresy; `relation_kind` musí rozlišit ZD od non-ZD, comm, assoc, alt, shared a birth. Tento klíč je strukturální adresa algebraické identity, nikoli automatické uložení jejího sémantického obsahu.

2. Nulové matice neukládat; aplikovat je

Zděděná incidence $A_t = A_0 \otimes J_m$, $m = 2^t$, umožňuje aplikaci na rozdělený vektor $x_{p,u}$ bez jediného alokovaného $6m \times 6m$ bloku:

$$s_q = \sum_{v=1}^{m} x_{q,v}, \quad r_p = \sum_q (A_0)_{pq} s_q, \quad y_{p,u} = r_p. \tag{1}$$

Náklad: obecná dense aplikace $O((6m)^2)$, generativní aplikace $O(6m + 6^2)$, paměť $O(6 + m \cdot 6)$ pro vstup a výstup, $O(6)$ pracovní sumy. V případě nematerializovaného/streamovaného výstupu lze i výstup konstruovat lazy. Pro $m = 128$ by měla jedna šestikořenová matice 589824 explicitních prvků; generativní operátor uchovává pouze 6×6 kořen a velikost rodiny.

```
def apply_inherited(a0, x):
    # x[p][u], p in 0..root-1, u in 0..m-1
    sums = [sum(block) for block in x]
    out = [sum(a0[p][q] * sums[q] for q in range(len(a0)))
           for p in range(len(a0))]
    return [[out[p]] * len(x[p]) for p in range(len(a0))]
```

Pozor: jde o adjacency zděděné komponenty, nikoli o celou signed CD multiplikaci a nikoli o všechny born komponenty.

3. Inherited vs birth: jak reprezentovat skutečnou novost

Pro každé zdvojení uložíme: (a) kořenový nebo již známý zděděný faktor, (b) volací pravidlo sign rekurze, (c) **birth frontier**, který nese jen nově vzniklou kvalifikovanou vazbu, a (d) volitelné cached čtení. Kombinatorický budget splňuje $B_{2N} = E_{2N} - 4E_N$. Samotný počet však nevytváří seznam nových adres: generátor musí nové χ a čtyřadresový oracle skutečně procházet nebo jinak konstruktivně enumerovat. Výsledný runtime by měl umět vrátit nejen `is_zero` ale i `why`: provenience kořene, signed predikát, reprezentanta třídy a úroveň narození.

4. Algebraické indexy pro AI a stavové systémy

Genealogii používáme jako strukturovaný **algebraický index**, nikoli jako automaticky naučenou reprezentaci reálných dat. V AI, distribuovaném zpracování a aplikačních grafech oddělujeme matematický klíč, reálně zaznamenaný stav a sémantický obsah. Počty a signatury vztahů se dopočítávají algoritmem, skutečné uživatelské události a parametry modelu se ukládají.

Výpočetní pipeline uchovává `relation_kind` pro `ZD`, `non-ZD`, `comm`, `assoc`, `alt` a `birth`. Nový vstup může upravit existující větev nebo přidat samostatnou `birth` větev; matematické zdvojení samo o sobě nevytváří zkušenost, fakta ani naučené váhy. Operátory trojných součinů musí zachovat závorkování. Pro pozorovatelnost používáme hodnost Krylovovy matice a požadujeme další sondu právě tehdy, když chybí rozměry readoutu.

5. Paměťová pyramida: VRAM / RAM / NVMe

- **VRAM:** aktivní amplitudy, hot blocky, GPU batch pro ε, Krylovovy báze, lokální operátory a jejich dočasné projekce.
- **RAM:** ancestry index, hot/warm factor cache, recent birth frontier a prefetch plány.
- **NVMe:** committed stav, provenience, immutable kořeny, transakční log, nové sémantické objekty a kvalifikované ZD frontier.

Vrstvy přesouvají **data a aktivní support**, nikoli matematickou definici pléna. Čisté `K_{2048}` neimplikuje alokaci 2048^2 tabulky. Strukturované operátory mohou být implicitní; samotný živý stav však pro hustých 2048 amplitud stále vyžaduje příslušné amplitudy.

6. GPU paralelismus — rozdělovat podle algebraické práce

Efektivní kernelová hranice je typicky `chi × ancestry-prefix × relation_kind × active-support`. Pro zděděné segmenty použijeme redukce bloků a broadcast podle (1). Pro born segmenty frontier spustí řídký generátor nebo sparse gather. Pro výpočet signed násobení je přirozené seskupení XOR a bitové rekurze; paměťový layout je `struct-of-arrays` pro koalescentní GPU fetch. Neslučujeme různé vztahové kanály do jediného booleovského grafu.

```
stage A: select active (root, chi, ancestry, relation-kind)
stage B: classify inherited factor / born frontier
stage C: batch sign oracle + sparse algebraic operators
stage D: reduce partial responses deterministically
stage E: collect closure residual + persist qualified delta
```

Synchronizace je nutná tam, kde `assoc` nebo `alt` propojí odlišné χ-bloky. Deterministické sloučení musí uchovat orientaci, sign a závorkování.

7. Databáze a obecné stromy

Jde o dvě různé úrovně implementace: `source algebra index` pro matematickou sémantiku a `inspired hierarchical index` pro běžná data. U obecného dokumentového či aplikačního stromu nepředstíráme, že jeho podniková logika vznikla z CD násobení; přebíráme mechanismy strukturálního sdílení a delta reprezentace.

Návrh schématu PostgreSQL:

```sql
CREATE TABLE zd_root (
  root_id BIGINT PRIMARY KEY,
  level INT NOT NULL,
  chi BIGINT NOT NULL,
  scheme_version INT NOT NULL,
  immutable_hash BYTEA NOT NULL
);
CREATE TABLE zd_delta (
  root_id BIGINT REFERENCES zd_root(root_id),
  ancestry BYTEA NOT NULL,
  birth_level INT NOT NULL,
  a BIGINT NOT NULL, b BIGINT NOT NULL,
  c BIGINT NOT NULL, d BIGINT NOT NULL,
  relation_kind SMALLINT NOT NULL,
  orientation SMALLINT NOT NULL,
  payload_ref BYTEA,
  PRIMARY KEY(root_id, ancestry, a,b,c,d,relation_kind)
);
CREATE INDEX zd_delta_birth_idx ON zd_delta(birth_level, relation_kind);
```

Store only qualified novelty and actual semantic payload; inherited mathematically determined structure is calculated on reads. Transakční log zaručuje committed stav. Celočíselné adresy nad `BIGINT` vyžadují širší bitset/byte encoding.

8. Higher-π jako výpočetní orákulum a optimalizační metrika

Algebraické invarianty `V_N`, `Phi_N`, `Q_N`, `Q_phase`, `C_N` poskytují přesné jednotkové testy bez Monte Carla. `Q_N` kvantifikuje průměrný čtvercový defekt násobení normovaných zdrojových prvků; je to analytická charakteristika jednotlivé CD vrstvy, kterou lze použít pro numerické testování signed kernelů, kontrolu implementace a výběr reprezentace podle požadované přesnosti.

```python
from fractions import Fraction
from math import comb
```

```python
def edges(n):
    return 0 if n < 16 else (n-2)*(n-4)*(n-8)//16

def q(n):
    return Fraction(32*edges(n), n*n*(n+2)**2)

def q_phase(n):
    return Fraction(32*edges(n), (n-1)**2*(n+1)**2)

def compound(n):
    return Fraction(comb(n,n//2), 1) * (1+q(n))**2 / (1+q(2*n))

assert q(16) == Fraction(7,216)
assert q(32) == Fraction(315,9248)
assert compound(8) == Fraction(15120,223)
```

9. Reprodukovatelnost a výkonnostní kontrakt

Dodané: `code/src/main.rs`: rekurzivní signed-CD oracle, signed nulový predikát, explicitní svědek a Rust unit testy pro `K16`, `K32`, four-lift. Výpočty K16/K32/K64 a svědka byly nezávisle numericky ověřeny Python ekvivalentem. Rust kompilátor nebyl v tvorbě balíku dostupný; zdroj je připraven ke spuštění příkazy `cargo test` a `cargo run --release -- 32`.

Benchmark vůči CSR: pro stejné úlohy změřit (i) generování E_N, (ii) `apply(A,x)`, (iii) random ancestor lookup, (iv) born-frontier inkrement, (v) persist+reload, (vi) Krylov probes, (vii) celé end-to-end task. Uvádět peak RSS/VRAM, propustnost, p50/p99, přesnost a bajty na kvalifikovanou vazbu. Faktorované zděděné bloky mají algoritmickou výhodu; celkové produkční zrychlení se musí spočítat na konkrétní pracovní zátěži.

10. První výpočetní syntéza

ZD genealogie přináší výpočetní princip, ve kterém je **matematicky vynucený vztah algoritmem, nikoli položkou databáze**. Higher-π poskytuje přesné kontrolní invarianty téhož algebraického zdroje; born-frontier zachovává skutečnou novost a multiplex udržuje nonalternativní informaci. Výsledkem je uskutečnitelný základ pro rozlehlé implicitní paměťové a výpočetní světy, včetně stavové a AI systémy — ne přilepený ZD graf, ale řízená generativní operátorová topologie.

11. Úplný typovaný kontrakt a algebraické invarianty

Dělíme identitu algebraického objektu, jeho sémantický stav a fyzickou adresu uložení. `AlgebraKey` obsahuje `tower_version`, `level`, XOR barvu, dvě kanonizované dvojice bázových adres, relativní sign a `relation_kind`; `StorageKey` navíc kóduje `root_id`, ancestry, epochu a verzi schématu. `Semantic-Payload` je nezávislá content-addressed data struktura. Signed oracle nikdy nevytváří historickou zkušenost uživatele, pouze ověřuje algebraickou povolenost vazby.

Závorkování není metadata na okraji: strom výrazu `Mul(Mul(x,y),z)` se musí rozlišovat od `Mul(x,Mul(y,z))`. Při přesunu mezi CUDA a CPU musí referenční exact oracle ověřit znak, podporu výsledku i asociátor/alternátor. Pro floating-point amplitudy testujeme zvlášť numerickou odchylku; algebraický index sám čísla nezaokrouhluje.

12. Výpočetní jádra a nákladový model

Nechť root incidence má r uzlů, e nenulových hran a rodina má $m = 2^t$ následníků na root. Plný explicitní blok obsahuje r^2m^2 prvků. Generativní `apply` má cenu $\Theta(rm+e)$ pro redukci vstupu, sparse root aplikaci a materializaci výstupu; pracovní paměť kromě vstupu/výstupu $O(r)$. Pro náhodné čtení jednotlivé hrany stačí klasifikace root dvojice a ancestry, bez alokace celé matice.

Algoritmus pro řídký birthday frontier odděluje tři kroky: deterministickou enumeraci kandidátních assessorů, rozhodnutí $W = +1$ a deduplikaci výsledných klíčů. `birth_count` samotný nenahrazuje konstruktivní enumerátor. Při paměťové kompresi $\Theta(m^2) \to \Theta(1)$ jde o **reprezentaci jedné pravidelné komponenty**, nikoli o univerzální kompresní poměr všech dat či libovolného grafu.

13. Paralelní determinismus a persistence

Work units seskupujeme podle `(level, chi, root, ancestry_prefix, channel)` a přidělujeme jim monotonický `epoch`. Každá vrstva produkuje immutable `Delta{epoch, source_hash, op, payload_hash}`. Redukce musí mít jednoznačné pořadí při floating-point aritmetice; integer/symbolic výsledky mohou používat komutativní agregaci jen tam, kde to dovoluje algebraická operace. `commit` probíhá až po validaci všech rodičovských adres, externí sémantické payloady zůstávají obsahově adresované.

PostgreSQL transakčně ukládá *zápisy*, nikoli generativní pravidla přepočítávaná jako data. NVMe snapshot obsahuje epochu, verzi sign rekurze a root hash; cold reload provede recalculation oracle a ověří kontrolní součty. Multiplexní dotaz vrací `ZD`, `non-ZD`, `comm`, `assoc`, `alt`, `birth` odděleně, a pokud sondy neobsáhnou celý Krylovův prostor, uvádí přesný rozměr nepozorovatelného komplementu.

14. Výpočetní experiment a regresní protokol

Dodané `code/benchmark.py` provádí přesně stejný `apply` s explicitní referenční maticí a faktorizovaným blokem. Výstupy se kontrolují na rovnost a ukládají se jako CSV s typem experimentu, rozměrem, dobou a odhadovaným objemem materializovaných dat. Nepoužíváme naměřené doby jedné konkrétní platformy jako univerzální hardwarový závěr. `code/verify.py` certifikuje základní číselné identity; `code/src/main.rs` reprodukuje signed oracle nezávislou jazykovou implementací.

Učení a změnu kompetencí systému vyhodnocujeme třemi nezávislými testy: (a) zachování exaktní algebraické identity po refinementu, (b) správnost persistovaných pracovních a trvalých stavů, (c) změna skutečných kognitivních kompetencí v teacher-removed testu. Pouhé snížení VRAM není doklad učení a úspěšné algebraické testy nejsou dokladem obecné inteligence.

15. Závěr

Jednotný přínos je konkrétní: signed CD definice vytváří adresovatelné vztahy, genealogie umožňuje faktorizované implicitní výpočty, birth frontier zachycuje nové členy a normově-momentové invarianty jsou přesnými regresními kontrolami. Tento návrh je realizovatelný bez materializace celé CD tabulky; strukturální úspory jsou vyjádřeny algoritmickými náklady a doprovodným benchmarkem.

Literatura

[1] V. Bíža, *Fraktální výpočetní algebra dělitelů nuly — signed Cayley–Dicksonův informační les a vyšší π*, doprovodný Paper A.
[2] G. Moreno, *Constructing zero divisors in the higher dimensional Cayley–Dickson algebras*, arXiv:math/0512517.
[3] T. A. Davis, *Algorithm 1000: SuiteSparse:GraphBLAS*, ACM Transactions on Mathematical Software.